Knuth Morris Pratt Algorithm

AlgoExpert · Very Hard · Famous Algorithms

The problem

- Does `sub` occur in `str`?
- Prefix-function (failure table) matching.

Intuition

- A failure table tells us how far to fall back on a mismatch, so we never rescan the text.

Brute force vs optimal

- Naive matching → O(n·m)
- KMP → O(n + m)
- Failure table avoids re-scanning

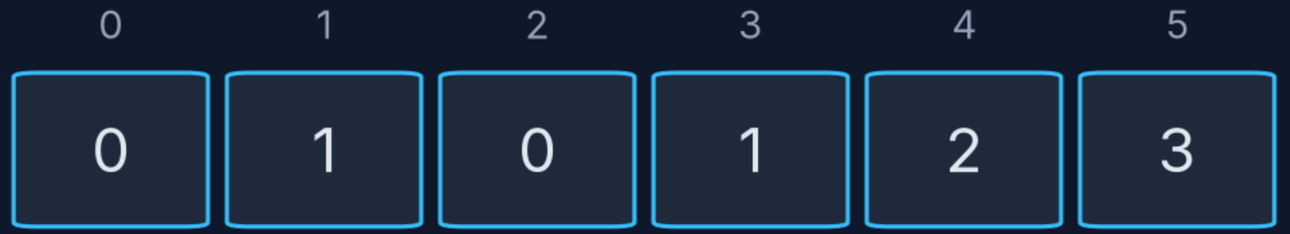
Approach

- 1. Build the prefix function of the pattern
- 2. Walk the text
- 3. On mismatch, fall back via the table
- 4. Match when the pattern is complete

Worked example



failure (longest proper prefix = suffix)



Build the failure table

Worked example

```
text: aabaaxaabaab

mismatch at x → jump using fail, don't restart
```

On mismatch, fall back via the table

Worked example

```
k reaches len(sub) → found ✓
```

Match found

Java solution

```java
public class KnuthMorrisPrattAlgorithm {
    public static boolean knuthMorrisPrattAlgorithm(String str, String sub) {
        if (sub.isEmpty()) return true;
        int[] fail = new int[sub.length()];
        for (int i = 1, k = 0; i < sub.length(); i++) {
            while (k > 0 && sub.charAt(i) != sub.charAt(k)) k = fail[k - 1];
            if (sub.charAt(i) == sub.charAt(k)) k++;
            fail[i] = k;
        }
        for (int i = 0, k = 0; i < str.length(); i++) {
            while (k > 0 && str.charAt(i) != sub.charAt(k)) k = fail[k - 1];
            if (str.charAt(i) == sub.charAt(k)) k++;
            if (k == sub.length()) return true;
        }
        return false;
    }
}
```

Complexity

- Time: O(n + m)
- Space: O(m)

Edge cases & pitfalls

- Empty pattern → true
- Build fail with the same loop
- Off-by-one in fail[k−1]

Interview tips & follow-ups

- Mention Z-algorithm / Rabin–Karp

Recap

- A failure table tells us how far to fall back on a mismatch, so we never rescan the text.
- Complexity: O(n + m)